



UltrON Platform Documentation

System Architecture, Remote Management & Licensing Operations Manual

Product Version: 1.0.70.5

Author / Owner: Neeraj (Developer, Sunshine Technologies)

Target System: Industrial Client PC & Central Server Management (RajAPI)

Compliance Standard: CPCB & SPCB Real-Time Telemetry Regulations

Date: July 2026

1. System Overview

UltrON is an industrial Control & telemetry platform designed by Sunshine Technologies. The system provides industrial plants with real-time Modbus register readings, SPCA/CPCB compliance monitoring, automated alert configurations, and dynamic LED board integrations. The platform operates on a split-architecture deployment model consisting of an on-site plant execution engine and a central administrator command interface.

1.1 Split Architecture Model

- **Client Side (Plant Location):** The onsite installation runs as a packaged Windows executable (UltrON.exe). It consists of a Python FastAPI backend daemon, an asynchronous Modbus register polling engine, a SQLite relational data store, and a React/Vite desktop dashboard frontend. It connects locally to analyzer units and displays LAN-based LED outputs.
- **Central Server (RajAPI Server):** Managed centrally at rajapi.com (running on a Raspberry Pi 5 platform). It aggregates historical fleet data, publishes broadcasts, pushes firmware OTA commands, and governs AMC client locks.

2. Licensing, Expiry & Lock Engine

To enforce commercial compliance and AMC protection, UltrON implements a non-intrusive, offline-safe licensing engine. It operates using the following lifecycle:

1. **Setup & Activation:** On first launch, if `CENTRAL_API_KEY` is absent in `.env.enc`, UltrON locks access and presents the Activation UI. Entering a valid site API key performs a one-time verification against `POST /api/v1/heartbeat` at RajAPI. Upon success, credentials encrypt, and initial lock state caches.
2. **Offline Resilience:** Once activated, the client does not need constant sync to remain licensed. It operates offline using the locally cached lock configurations in `client_lock.json`. The local system will only trigger an AMC expiration lock if the client PC system clock exceeds the cached `amc_expiry` date.
3. **Heartbeat Auto-Renewal:** When online, the background heartbeat worker pulls latest AMC dates. If renewed on RajAPI, the client cache automatically updates and unlocks without manual intervention.
4. **Technician Backdoor:** In case of internet failure or emergency lockout at a remote site, entering the technician passcode `Ultronpoi` triggers a temporary local override, restoring local GUI access for maintenance.

3. Telemetry Sync & Offline Queuing (FIFO)

For centralized data science, historical charting, and compliance auditing, client telemetry is automatically forwarded to RajAPI's database using the sync engine.

3.1 Core Sync Mechanics

- **Live Data (1-minute):** Real-time parameter values (e.g. concentrations, flow rates) are pushed every 60s during active live mode.
- **Averages (15-minute):** Regulatory 15-minute averaged values are synced during delayed upload loops.
- **Failure Queuing:** If RajAPI is offline or network connection is lost, telemetry pushes fail. Instead of discarding the data, the payloads are buffered in the SQLite pending_uploads table.
- **Chronological Backfill (FIFO):** The background daemon checks for connection recovery. Once online, it uploads queued payloads in strict chronological order (ordered by created_at ASC) using X-API-Key headers, backfilling the central database without leaving timeline gaps.

4. Regulatory Compliance & Isolation

CPCB compliance requires strict isolation between different communication tasks to prevent single-point failures from halting transmission:

Service	Type	Protocol	Description / Isolation Guarantee
RajAPI Sync	Centralized	HTTPS / JSON	Aggregates fleet data for Neeraj's dashboard and data analysis. Completely asynchronous.
SPCB Push	Regulatory	HTTPS / JSON	Pushes telemetry to State PCBs. Uses separate config tables in SQLite.
CPCB Export	Regulatory	Local File	Creates daily Annexure-I flat CSV files locally on disk. Immune to internet timeouts.
LED Board	Local Display	HTTP / JSON	LAN API endpoint (/api/v1/led) reads locally cached LiveData. Runs 100% offline.

4.2 Compliance Quality Codes

All telemetry packets follow CPCB regulatory standard quality codes representing parameter health:

Code	Definition	Usage Scenario
U	Valid Data (Good)	Normal operating conditions; sensor reading is stable and calibrated.
O	Out of Range	Sensor reads values exceeding maximum calibrated limits.

E	Error / Calibration	Sensor is undergoing maintenance, calibration, or registers a fault.
N	No Data	Connection lost between the PC and the physical analyser.

5. Performance & Safety Safeguards

To ensure that adding remote diagnostic heartbeats and telemetry pushes does not degrade the performance of UltrON.exe or cause system crashes at client plant PCs, the following design precautions are enforced:

- 1. Out-of-Band Async Execution:** Heartbeat checkins, telemetry uploads, and pending retries execute exclusively in asynchronous threads via FastAPI's event loop and APScheduler. The primary register polling loop remains unhindered, guaranteeing zero lag in Modbus data ingestion.
- 2. Connection Dropouts Protection:** All outbound connections set a strict 5.0s connection timeout. If RajAPI is slow or down, threads release instantly, preventing process backlog.
- 3. Database Lock Prevention:** Reuses randomized backoff retries and write semaphores implemented in v1.0.69 to safeguard the SQLite DB against write lock contentions between the polling loop and retry daemon.
- 4. Memory-Safe Diagnostic Fetching:** Log tail queries (e.g. `fetch_logs` commands) open files remotely using a tail-seek pointer to load only the last 150 lines, capping RAM consumption at <50 KB.